



SAP Labs India – Placement Experience

GURUMOORTHY R
CSE '26

About the Company:

SAP Labs India is a key innovation and R&D hub for SAP, with a global presence in five cities: Bengaluru, Gurgaon, Pune, Mumbai, and Hyderabad. Bengaluru hosts the largest campus, serving as a crucial center for driving AI solutions and enterprise technology development for global customers. The labs focus on a customer-centric and agile approach to innovation, covering user experience, business models, and cutting-edge technology.

SAP (short for Systems, Applications, and Products in Data Processing) is one of the biggest names in enterprise software, helping businesses around the world run smarter, whether it's managing their supply chains, handling finances, or making sense of their data. Headquartered in Germany, SAP has been around for over 50 years and works with companies across almost every industry you can think of (they are pretty much omnipresent - being a b2b company integrated with almost every major company today). For students and early-career professionals, it's a great place to explore both tech and impact at a global level.

Selection Process:

Online Assessment (OA):

280 students were shortlisted for the online assessment. OA was conducted at IT department Lab. Time Duration for OA is **100 minutes** (1 hour 40 mins). Platform – **Codility Platform**. Both video and audio proctoring were there.

Totally **2 DSA questions** were asked. Everyone got **different** set of questions. One question was easy level and another one was medium level.

Questions asked for me:

1.Number of ways to split a string

Given a string **s** consisting of **N** lowercase English letters. In how many ways we can split **s** into 2 non-empty parts such that in at least one part the letter **x** and the letter **y** occur same number of times. Return number of splits.

Example 1:

s = "ayxbx"

Output: 3

Explanation:

Split 1: a/yxbx -> first part "a" has equal occurrence of x and y (x = 0, y = 0)

Split 2: ayx/bx -> first part "ayx" has equal occurrence of x and y (x = 1, y = 1)

Split 3: ayxb/x -> first part "ayxb" has equal occurrence of x and y (x = 1, y = 1)

No other valid split is there.

So, output is 3.

More Examples:

s = "xzzzy"

Output: 0 (Because we can't split into 2 non-empty strings such that in at least one part the letter **x** and the letter **y** occur same number of times)

s = "toyxmy"

Output: 5

Splits: t/oymxmy, to/yxmmy, toy/xmmy, toyx/my, toyxm/y

2. Balance the given the board

There is a board made of **2 rows** and **N columns** represented by two strings **row1** and **row2**.

Made of characters **R** or **W** or **?**.

Given board is **Balanced** if in each row & column, Number of R is equal to number of W.

row1 = ?RW?WR

row2 = ?WR?RW

?	R	W	?	W	R
?	W	R	?	R	W

You can see that, in each row and column, no of R and W are same.

So, this board is balanced.

? can be replaced by W or R.

What is the **minimum number of replacements** needed to balance the board?

If not possible, return -1.

Example 1:

row 1 = W?WR?, row2 = R??W?

Output: 3

Explanation:

W	?	W	R	?
R	?	?	W	?

We can convert the board into (i) or (ii) so that board is balanced:

(i)

W	?	W	R	R
R	?	R	W	W

Number of replacements = 3

(ii)

W	R	W	R	?
R	W	R	W	?

Number of Replacements = 3

More Examples:

row1	row2	Output
R?R??	??W??	5
RR?	?R?	-1
WWW??	RRR??	-1
?RW?WR	?WR?RW	0

A total of **89** students were shortlisted for the interview. The results were announced just one day before, in the afternoon, which meant I had only from that afternoon until the next morning to prepare. From this experience, my key takeaway is that you should start preparing for interviews alongside your coding round preparation. This way, even if the results are announced at the last moment, you will already be confident and well-prepared for the interview.

Interview:

There were 4 Rounds of interview. It was conducted in person at CUIC.

1. Technical Round 1
2. Technical Round 2
3. Techno-managerial Round
4. HR Round

Round 1 – Technical Round 1:

Duration: 50 mins

Panel: 1 interviewer

1. Introduction & Project Discussion

- The interviewer started with: *“Introduce yourself.”*
- Next, he asked me to explain one of the projects I am most familiar with.
- Since I mentioned being deeply interested in **Web Development**, he asked:
 - What is Web Development?
 - What are its components? (Front-end, Back-end, Database, etc.)

2. Database & Web Development Question

Scenario:

I was asked to design a form consisting of **Name, Skills, and Submit button**.

This form is used by CUIC: when a company arrives for placement, we need to return students whose skills match the company's requirements.

My Approach:

- I explained that **Skills is a multivalued attribute**, so it needs to be stored in a separate table.
- For unique identification, I added **Roll No** to the form.
- I designed the database schema and wrote queries to retrieve the students with matching skills.

3. DSA Question – Graphs

Problem Statement:

There are N number of files with dependencies given as A -> B (file A depends on file B).

From the given dependencies, find the order of selecting/executing the files.

My Solution:

- I identified this as a **Topological Sort problem**.
- Explained the approach step by step (BFS/DFS method).
- Wrote the code clearly while explaining my thought process.

- Important Tip: *Even if you know the solution, take time to think, explain clearly, and structure your answer logically.*

The interviewer was impressed with my explanation and coding.

4. Project Discussion (Again)

- We had a short discussion about my projects again.
- He appreciated the way I explained them.
- My project explanation created a positive impression and led to further discussion.

5. Final Interaction – Candidate's Questions

- At the end, the interviewer asked: *"Do you have any questions for me?"*
- This is the best chance to show your **interest and willingness**. Don't skip it. Ask meaningful questions (about role, team culture, learning opportunities, etc.). It creates a positive impression.

Overall, this round tested my ability to explain projects, apply database knowledge, solve DSA problems, and communicate effectively. Clear explanation and confidence played a key role.

After this round, I was **shortlisted for the next stage**. Out of all the participants, around **25 students** were shortlisted for **Technical Round 2**.

Round 2 – Technical Round 2:

Duration: 1 hour 15 mins

Panel: 2 interviewers

The round began with a short introduction from both sides. I was asked to briefly explain one of my projects, after which the interview quickly moved into DSA-focused questions.

Question 1: Searching order with Ranking

Problem Statement:

- A set of words is given, each with a rank.
- When a user types in the search bar, all words matching the **prefix** of the search key should be displayed.
- The results should be ordered by **rank (descending)**.

Example:

Words with rank:

app (10), application (8), apple (9), apply (5), banana (15)

Search Key = "app"

Result = app (10) → apple (9) → application (8) → apply (5)

(Note: "banana" is ignored since it does not match the prefix "app")

My Thought Process & Approach:

- Since the problem deals with **prefix matching**, the first idea that came to my mind was to use a **Trie data structure**.
 - Each word can be inserted into the Trie.
 - Searching for the prefix is efficient ($O(\text{length of prefix})$).
- After finding the prefix node in the Trie, we need to return all words under that prefix **sorted by rank**.

Clarification I Asked:

- I asked the interviewer: *"If there is a word like banana(15), should it appear when the search key = app?"*
- The interviewer said **No**, since only prefix matches are valid.
- That confirmed my Trie-based approach was correct.

Optimization Discussion:

- My initial idea: Insert all words in the Trie → When searching, collect words under the prefix → Sort them by rank before displaying.
- Interviewer expectation: Further optimize.
 - Sorting every time can be costly if there are many words.
 - Possible optimization: At each Trie node, maintain a **Heap (Max-Heap or Min-Heap)** or a **pre-sorted list** of top words for quick retrieval.
- We discussed how using a **Heap along with Trie** could give better performance for large datasets.

Question 2: Infection Spread in Cities (Grid + BFS)

Problem Statement:

- A grid is given where each cell represents a **city**.
- Each cell has two possible states:
 - 1 → City is already infected
 - 0 → City is not infected
- Infected cities spread the infection to their **adjacent (up, down, left, right)** cities in **1 day**.
- The task is to determine **the day on which all the cities become infected**.

My Approach & Explanation:

- I quickly recognized this as a variation of the **Rotten Oranges problem**(<https://leetcode.com/problems/rotting-oranges/>)
- The natural approach here is to use **Breadth-First Search (BFS)**:
 - Treat all initially infected cities (1) as **sources** and push them into a queue with day = 0.
 - Perform BFS to spread infection level by level (day by day).
 - Track the maximum day reached when the last city becomes infected.

Code Writing:

- The interviewer asked me to write the code.
- I implemented the BFS solution step by step:

1. Traverse grid → push all 1 cells into queue with day = 0.
2. BFS → spread to adjacent cells and mark them infected.
3. Keep track of days.
4. Return the maximum day.

Edge Cases (Discussed with Interviewer):

1. **No city is infected initially** → Answer should be 0 (no spread possible).
2. **All cities are already infected** → Answer should be 0 (no extra days needed).
3. **Some cities can never be infected (isolated regions with no adjacent infected cities)** → Should return -1 or handle accordingly.

Question 3: Designing a Stack with Min/Max Operations

Problem Statement:

- Design a **Stack** that supports the following operations:
 - push(x) → Add element at the top of the stack.
 - pop() → Remove the element at the top of the stack.
 - getMin() → Return the minimum element present in the stack.
 - getMax() → Return the maximum element present in the stack.
- The stack should efficiently handle MinElement and MaxElement queries.

My Approach & Explanation:

1. Brute Force Approach (Initial Idea):

- Use the standard **stack container in C++** for push and pop.
- For `getMin()` and `getMax()`, traverse the entire stack to find min/max.
- Simple but inefficient → $O(n)$ time for min/max queries.

2. Optimized Approach (Inspired by MinStack Problem):

- Store each element as a **pair** (a, b), where:
 - a = element itself
 - b = minimum element in the stack till now
- Similarly, maintain another stack (or extend pair) to track **maximum element** till now.
- This way, push, pop, `getMin()`, and `getMax()` all work in **$O(1)$** time.

Discussion with Interviewer:

- I explained both brute force and optimized approaches.
- We discussed **pros and cons**:
 - Brute force → Easy to implement but slow for large inputs.
 - Optimized (with pairs) → Extra space usage but very efficient.
- The interviewer appreciated that I explained the **trade-offs** and thought about scalability.

Database & DBMS Discussion

1. Initial Questions:

- The interviewer asked: *"Are you strong in DBMS? Have you ever worked on projects involving DBMS?"*
- Since I had mentioned **MySQL** and **MongoDB** in my resume, I answered confidently "Yes".

2. SQL Queries:

- I was asked to write some **basic SQL queries** (CRUD operations, simple joins, filtering, etc.).
- Then they asked me about **Triggers** and **Stored Procedures**:
 - **What is a Procedure?**
 - **Why do we need it?**
- I explained the concept:
 - **Procedures** are reusable blocks of SQL code stored in the database.
 - They help in **modularity, reusability, and better performance**.

3. Project-Based Question (Concurrency Handling):

- From my project experience, they asked:
"How will you handle concurrent access to the database?"
- I explained using **locking mechanisms** in DBMS:
 - Locks ensure that when one transaction is accessing/updating a data item, others wait until it is released.

4. Follow-up Question (Lock Failure Case):

- The interviewer went deeper:
"If someone acquires a lock for a particular data item (say X), but during execution the process fails, how will you release the lock?"
- I initially explained using **rollback functions**, but I was a bit confused here.
- I tried my best to answer, but I wasn't fully confident.

API Discussion

1. Questions Asked:

- *"What is an API?"*
- *"How will you write an API in Java?"*
- *"Write some example APIs in Java."*

2. My Response:

- I explained the concept of **API** (Application Programming Interface) as a way for two systems to communicate.
- Though I was not very familiar with writing APIs from scratch, I managed to answer using my knowledge of **Spring Boot** and annotations like `@RestController`, `@GetMapping`, etc.
- The interviewers were satisfied with my explanation.

Interviewer's Feedback & Soft Skills

- Both interviewers appreciated **the way I explained my answers** and highlighted my **confidence** and **eye contact** during the discussion.
- **Key Point:** Confidence plays a huge role. Even if you don't know the exact answer:
 - Take a moment, think logically, and explain what you know.
 - If still stuck, politely ask the interviewer for hints — they are often ready to guide you.

Final Interaction

- At the end, they asked me: *"Do you have any questions for us?"*

- As I mentioned earlier, this is a **very important opportunity**. I spent nearly **15 minutes** asking meaningful questions about the role, projects, and company culture.
- The interviewers appreciated my curiosity and concluded with: *"It was a very wonderful interview with you, Gurumoorthi."*
- I received a **positive outcome** from this round. I was **selected for the next round**.

Round 3 – Techno-managerial Round:

Duration: 1 hour 20 mins

Panel: 2 interviewers (Managers)

This was one of the most important rounds in the whole selection process, as it was completely focused on **real-world application-oriented questions** and tested my **practical knowledge**.

As usual, the round started with my introduction, followed by the interviewers introducing themselves.

They began with a set of technical and conceptual questions:

- Difference between **Virtual Machines and Containers**.
- **Concurrency vs Parallelism**, and to give real-world examples.
- The complete process behind **what happens when we enter a URL in a browser**.
- Difference between **HTTP and HTTPS**.
- What is a **RESTful API** and the important concepts behind it.
- For **CRUD operations**, what API methods are used.
- Asked me to **write some APIs** for different use cases.
- How to handle **dynamic API routes**.
- What is a **Socket** and its uses.
- Questions about **new technologies** I am currently learning.
- My understanding of **Docker and Kubernetes**.

This round was quite challenging as it connected **theoretical knowledge with practical scenarios**. I tried to explain with real-life examples wherever possible, and also connected some answers with my projects to show practical relevance.

The interviewers were very interactive throughout, and they often asked follow-up questions to test my depth of knowledge. I stayed confident and took a moment to think before answering whenever I needed.

After the technical and conceptual questions, they asked me to **explain any one of my projects** and specifically questioned **how I would handle concurrency in that project**. I explained the approach and related it to database concepts and application-level handling.

Then, the interview turned more interesting as they moved into **puzzle-solving**. They asked me a total of four puzzles:

1. **Missing Number Problem** – Given numbers from 1 to N, one number is missing. I explained the mathematical solution using the sum of first N natural numbers.
2. **8 Balls Puzzle** – Among 8 balls, one ball is lighter. Using a weighing machine at most two times, we need to find the odd ball. I solved it logically by grouping and comparing.
3. **Sand Clock Puzzle** – Two sand clocks of 11 and 7 minutes are given. We need to measure 15 minutes (not necessarily continuous). I explained the step-by-step approach to achieve this.
4. **Trains and Bird Puzzle** – Two trains start 80 km apart, moving towards each other at 40 km/h. A bird flying at 100 km/h keeps traveling between them until the trains meet. They asked me how many times the bird travels back and forth. I tried my best to solve this but due to time constraints, they moved forward.

I was able to **solve the first three puzzles completely**, and for the fourth, I explained my thought process even though I couldn't reach the final answer within the given time. The interviewers appreciated my logical approach.

Towards the end, they asked some **personal and preference-based questions** like:

- Something about my family.
- Which domain I prefer to work in (Frontend, Backend, Testing, etc.).
- How quickly I can learn new technologies.
- If I have visited Bangalore or other cities apart from Chennai.

Finally, they concluded the round with positive feedback. It was a **really nice and interactive interview**, and I felt both challenged and encouraged throughout. When I stepped out of the room, they immediately asked me to proceed to the **final round of interviews**.

It was a moment of **excitement mixed with nervousness**, but I gathered myself and entered the room for the final round with confidence.

Round 4 –HR Round:

Duration: 20 mins

Panel: 1 interviewer (Hiring Manager)

This was the **final round** of my interview process. As usual, it started with a brief introduction.

The HR asked me a few questions to understand my goals and alignment with the company:

- Why do you want to join SAP?
- Do you have any plans for higher studies?
- What do you know about SAP?
- What are you currently learning?

Finally, he asked me if I had any questions for him. I made sure to use this opportunity meaningfully. I asked:

- Why did you join SAP?
- What was your previous experience?
- Can you tell me about the growth opportunities for freshers at SAP?

It turned into a very nice discussion. The HR explained about the **work-life balance at SAP** and shared some of his personal experiences. I also asked how he manages his time and about the **new SAP campus at Devanahalli**, which led to an engaging conversation.

By the end of this round, I felt it was more of a friendly discussion than a formal interview.

Finally, after all the rounds were completed (around **8 PM in the evening**), the results were announced. I was thrilled to hear that **I was one of the selected students**. It was truly a very happy and proud moment for me.

That entire day taught me a lot—not only technical and problem-solving skills but also the importance of confidence, communication, and presence of mind in interviews.

I am very happy to share this whole experience.

Tips & Notes:

For Coding Round:

- **Confidence is key** → Don't panic if you get stuck, break the problem into smaller parts.
- **Attempt all questions** → Even partial code can fetch marks, so don't leave questions blank.
- **Start with Brute Force** → First write a correct solution, then optimize step by step.
- **Think about Edge Cases** → Empty input, single element, large constraints, negative values, etc.
- **Practice previous year questions** → Get an idea of the difficulty level and common patterns.

- **Time Management** → Don't spend too long on one problem, move to the next and come back later.
- **Dry run your code** → Before submitting, test with small cases manually.
- **Stay calm during OA** → Focus more on accuracy first, then optimization.

For Interviews:

- **Be confident** – Confidence is the most important quality throughout the interview process.
- **Revise your resume** – Before every round, go through what you have mentioned in your resume. You should be able to explain every point with clarity.
- **Research the company** – Be curious about the company and the role you are applying for. This shows your genuine interest.
- **Show eagerness to learn** – Even if you don't know everything, show that you are a quick and eager learner.
- **Have a good discussion** – Don't just answer; interact with the interviewer. Try to connect with them by asking meaningful questions about their work and experience.
- **Think before answering** – When a question is asked, take a moment to think. Then explain your answer clearly and confidently.
- **Handle unknown questions smartly** – Even if you don't know the exact answer, try to say something related to the question and show your thought process. Interviewers often value the way you think.

✨ **All the very best to you!**

Remember – **CONFIDENCE IS THE KEY TO SUCCESS.**

Thank you!

You can reach out to me through the contact details given below.

Gurumoorthi R

Mobile/ WhatsApp: +91 7845327200

Mail: gurumoorthirajakumar11@gmail.com

Linked In: <https://www.linkedin.com/in/gurumoorthi-r-ceg2022/>